Cybersecurity Experiment Guide: Username Enumeration

This document contains all the commands and scripts used in the experiment, arranged in the correct sequential order. All previous errors (such as Kali Linux repositories and externally-managed-environment) have been fixed here.

Phase 1: Fixing the System and Installing Prerequisites

1. Fix the Kali Linux Repositories (Resolves 'Unable to connect to mirror'):

```
echo "deb http://http.kali.org/kali kali-rolling main contrib \nnon-free non-free-firmwa
```

2. Update the system and install required tools:

```
sudo apt update
sudo apt install python3-venv redis-server -y
```

3. Start and enable Redis database:

```
sudo systemctl start redis-server
sudo systemctl enable redis-server
```

Phase 2: Setting up the Virtual Environment (Resolves 'externally-managed-environment')

1. Create the experiment directory and navigate into it:

```
mkdir -p ~/attack_exp && cd ~/attack_exp
```

2. Create and activate the virtual environment:

```
python3 -m venv venv
source venv/bin/activate
```

3. Install the required Python libraries inside the isolated environment:

```
pip install fastapi uvicorn redis httpx
```

Phase 3: Setting up the Defender Server

Create a file named main.py:

```
nano main.py
```

```
import os
import time
import asyncio
from fastapi import FastAPI, Request, Response
import redis.asyncio as redis

app = FastAPI(title="Instagram-like Security System")

redis_host = os.getenv("REDIS_HOST", "localhost")
redis_client = redis.Redis(host=redis_host, port=6379, db=0, decode_responses=True)

MAX_REQUESTS = 5
WINDOW_SIZE = 30
BAN_TIME = 60

async def is_rate_limited(ip: str) -> bool:
    current_time = int(time.time())
    key = f"limit:{ip}"

    async with redis_client.pipeline(transaction=True) as pipe:
        pipe.zremrangebyscore(key, 0, current_time - WINDOW_SIZE)
        pipe.zadd(key, {str(current_time): current_time})
        pipe.zcard(key)
        pipe.expire(key, WINDOW_SIZE + 10)
        results = await pipe.execute()

    return results[2] > MAX_REQUESTS

@app.get("/api/v1/check_username")
async def check_user(request: Request, username: str, response: Response):
    client_ip = request.client.host

    if await is_rate_limited(client_ip):
        print(f"[ALERT] Rate limit exceeded for IP: {client_ip}")
        await asyncio.sleep(4)
        return {"status": "taken", "message": "Security Check: Verified"}

    rare_usernames = ["vip", "king", "root", "admin", "kali"]

    if username in rare_usernames:
        return {"status": "taken", "available": False}

    response.status_code = 404
    return {"status": "available", "available": True}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=5000)
```

Run the defender server:

```
uvicorn main:app --port 5000
```

Phase 4: Setting up the Attacker Script

Open a NEW terminal, activate the environment, and create attacker.py:

```
cd ~/attack_exp
source venv/bin/activate
nano attacker.py

import asyncio
import httpx
import time

async def attack(username):
    url = "http://127.0.0.1:5000/api/v1/check_username"
    async with httpx.AsyncClient() as client:
        start = time.time()
        try:
            resp = await client.get(url, params={"username": username})
            print(f"User: {username:10} | Code: {resp.status_code} | Time: {time.time()-
        except: pass

async def start_flood():
    print("Starting fast enumeration attack (15 requests)...")
    tasks = [attack(f"user_{i}") for i in range(15)]
    await asyncio.gather(*tasks)

if __name__ == "__main__":
    asyncio.run(start_flood())
```

Run the attacker script:

```
python attacker.py
```